Pointers

We use * at declaration to specify a pointer variable.
We use & with a variable name to get its address.
When using pointers, * means get the value at the address the pointer points to

General syntax for declaration and initialization of variables vs pointers:
Variables: Pointers:

The value for an integer variable is always:

The value of a pointer is always:

Assume that the following valid statements are executed one after another in the given sequence. Update the values of the variables that change after each statement has executed.

```
int a{4};              // assume a is stored at address 0x1000
int b{8};              // assume b is stored at address 0x1004
int* ptr1{&a};         // assume ptr1 is stored at address 0x2000
int* ptr2{&b};         // assume ptr2 is stored at address 0x2008
int** ptr3{&ptr2};     // assume ptr1 is stored at address 0x2010

*ptr1 = 5;

**ptr3 = 10;

*ptr3 = ptr1;

*ptr2 = 12;

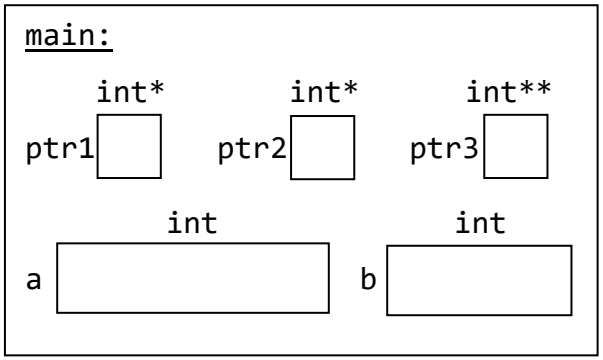
**ptr3 = *ptr2 + **ptr3;

ptr1 = &b;

ptr3 = &ptr1;

*ptr2 = **ptr3 + 1;

*ptr1 = a + 5;
```



Variable	Address	Value
a	0x1000	
b	0x1004	
…	…	
ptr1	0x2000	
ptr2	0x2008	
ptr3	0x2010	

References

Given the Time struct below, complete the add_time functions, which adds the given number of minutes to the t1 Time object.

```
struct Time {
  int hours;
  int minutes;
};
```

```
int main() {
   Time t1{13, 45};
   add_time_X(t1, 90};
   // t1 is now 15:15?
   return 0;
}
```

```
/**
 * @brief Modifies the given time by adding minutes to it
 * @param t the time to modify
 * @param minutes the number of minutes to add to it
 */
void add_time_copy(Time t, int minutes) {



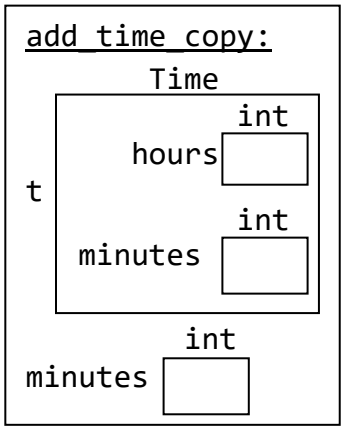
}

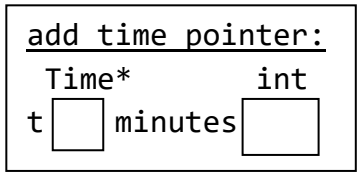
void add_time_pointer(Time* t, int minutes) {

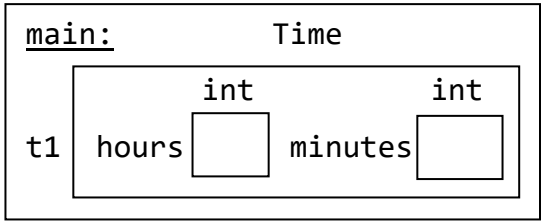

}

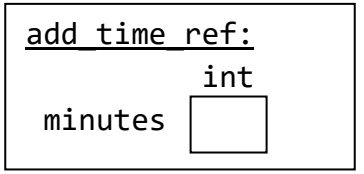
void add_time_ref(Time& t, int minutes) {


}
```





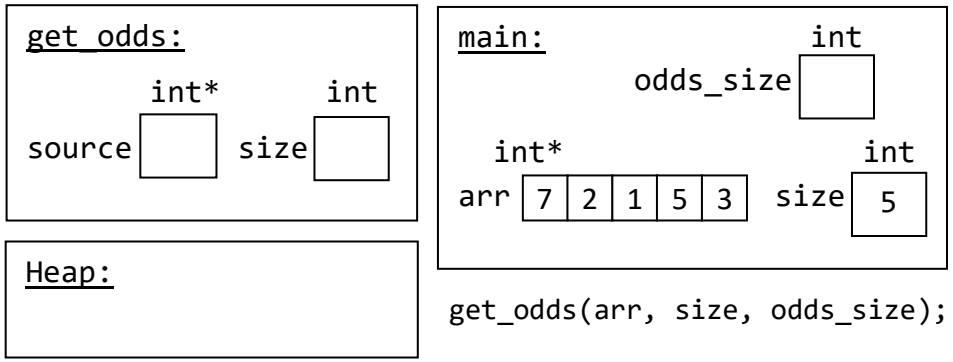




Dynamic Memory

```
/**
 * @brief Allocates a new array and copies odd values into it
 * @param source the array to copy from
 * @param size the length of source
 * @param new_size the length of the new array holding old values
 * @pre source is not null (but may be empty)
 * @return int* a new dynamically-allocated array containing a
 *         copy of source's odd values. Caller is responsible for
 *         freeing this array with delete[] when no longer needed
 */
int* get_odds(const int* source, const int size, int& new_size) {
```

```
}
```




```
/**
 * @brief Inserts new_value into the correct position in the
 *        sorted array, growing it by one element
 * @param array reference to the array pointer; this array is
 *        reallocated to make it point to a new, larger array
 * @param size reference to the current number of elements in
 *        source. Updated to size + 1 on return
 * @param new_value the value to insert
 * @pre array is a dynamically-allocated array of size elements
 * @pre array's is sorted in ascending (non-decreasing) order
 * @post the array originally pointed to by array has been freed.
 *       array now points to a new array of size + 1 elements,
 *       identical to the original but with new_value inserted
 *       into the correct position so it remains sorted.
 *       size has been updated to match the array's new length
 */
void insert_in_place(int*& array, int& size, int new_value) {
```

```
}
insert_in_place(arr, size, 3);
```

